

A binary semaphore library

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Συναρτήσεις βιβλιοθήκης

Κατασκευάσαμε τις εξής συναρτήσεις ως “API” της βιβλιοθήκης:

Συνάρτηση	Χρήση
<code>int mysem_init(mysem_t *s, int n);</code>	Αρχικοποίηση σηματοφόρου με τιμή n. Επιστρέφει 1 για επιτυχία, 0 αν το n
<code>int mysem_down(mysem_t *s);</code>	Μείωση σηματοφόρου κατά 1. Επιστρέφει 1 για επιτυχία ή -1 αν ο σηματοφόρος δεν έχει αρχικοποιηθεί.
<code>int mysem_up(mysem_t *s);</code>	Αύξηση σηματοφόρου κατά 1. Επιστρέφει 1 για επιτυχία, 0 αν ο σηματοφόρος είναι ήδη 1 ή -1 αν ο σηματοφόρος δεν έχει αρχικοποιηθεί.
<code>int mysem_destroy(mysem_t *s);</code>	Καταστρέφει τον σηματοφόρο. Επιστρέφει 1 για επιτυχία ή -1 αν ο σηματοφόρος δεν έχει αρχικοποιηθεί.

Υλοποίηση βιβλιοθήκης

Η βιβλιοθήκη υλοποιεί δυαδικούς (binary) σηματοφόρους με χρήση γενικών σηματοφόρων του System V. Στηρίζεται στο ότι όταν καλούμε την `mysem_up()` δεν αυξάνουμε την τιμή του σηματοφόρου IPC V πάνω από 1, παρόλο που αυτός είναι γενικός.

Multithreaded prime calculation with binary semaphores

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Αναλογία με παραγωγό-καταναλωτή

Παρατηρούμε άμεσα ότι πρόκειται για πρόβλημα ενός παραγωγού και πολλών καταναλωτών με buffer μεγέθους ενός στοιχείου.

Τα race conditions που παρατηρούμε είναι ότι ο κάθε καταναλωτής πρέπει να λαμβάνει μία μοναδική τιμή για να επεξεργαστεί και να μη χάνονται ενδιάμεσες τιμές.

Για να αποφύγουμε τα race conditions χρησιμοποιούμε τους δυαδικούς σηματοφόρους μας.

Λειτουργία main

Το main thread λειτουργεί ως παραγωγός. Διαβάζει τιμές από την είσοδο stdin και όσο δεν ανιχνεύσει EOF δίνει την τιμή που διαβάζει στον επόμενο διαθέσιμο worker, χρησιμοποιώντας σηματοφόρους για να αποφευχθούν τα race conditions.

```
temp = 0;
while(scanf(temp) != EOF)
{
    down(empty);
    down(mtx);
    worker_value = temp;
    up(mtx);
    up(full);
}
on = 0;
up(mtx);
up(full);
```

Λειτουργία του worker thread

To worker thread λειτουργεί ως καταναλωτής:

```
while(on)
{
    down(full);
    if(!on)
    {
        up(full);
        break;
    }
    down(mtx);
    temp = worker_value;
    up(mtx);
    up(empty);
    if(isPrime(temp))
        printf("Is prime");
    else
        printf("Is not");
}
```

Semaphore bridge

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Περιγραφή λύσης

Ο στόχος είναι να διέρχονται οχήματα έτσι ώστε:

- Να μην υπάρχει περίπτωση δύο οχήματα αντίθετης κατεύθυνσης να βρίσκονται πάνω στη γέφυρα,
- Να μην υπάρχει λιμοκτονία δηλαδή ένα αυτοκίνητο να μην μπορέσει να περάσει ποτέ τη γέφυρα
- Να μην υπάρχουν πάνω στη γέφυρα παραπάνω οχήματα από τη χωρητικότητά της.

Επιτυγχάνουμε αμοιβαίο αποκλεισμό με έναν σηματοφόρο `mtx` και κάνουμε `signal` τα αυτοκίνητα κάθε κατηγορίας με δύο άλλους σηματοφόρους `go left` και `go right`.

Περιγραφή λύσης

Ένα αυτοκίνητο επιτρέπεται να περάσει τη γέφυρα όταν:

- Γέφυρα κενή και είναι η σειρά του
- Γέφυρα έχει αυτοκίνητα ίδιου τύπου (αλλά δεν είναι γεμάτη)

Ένα αυτοκίνητο, περνώντας τη γέφυρα ελέγχει την ουρά των απέναντι αυτοκινήτων και την ουρά της δικής του πλευράς και αναλόγως λαμβάνει αποφάσεις για το ποιόν θα ξυπνήσει (αν ξυπνήσει κάποιον).

Για να βεβαιώσουμε την «δικαιοσύνη» στη λύση μας αλλάζουμε τη σειρά κατεύθυνσης όταν περάσουν το πολύ N αυτοκίνητα μιας κατεύθυνσης, όπου N το μέγεθος της γέφυρας.

Λειτουργία car thread

To car thread ελέγχει την κατεύθυνσή του και αναλόγως περνά ή όχι την γέφυρα.

```
if(bridge has same type cars AND it is our turn OR bridge is empty)
    pass the bridge
else if(bridge has different type cars OR bridge has same type cars and is full OR it isn't
our turn)
    don't pass (block)
sleep(time to pass)
awake (or not) next cars depending on turn, "fairness" counter, left queue and right queue
values
```

Λειτουργία main thread

Η main thread παράγει αυτοκίνητα. Δεν διαχειρίζεται η ίδια τα αυτοκίνητα ούτε ελέγχει το μπλοκάρισμα των σηματοφόρων.

Όλη η επικοινωνία για το ποιος θα περάσει γίνεται μεταξύ των ίδιων των αυτοκινήτων.

Σας ευχαριστούμε!

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Τρενάκι

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Λειτουργία προγράμματος

Το train thread προσπαθεί να γεμίσει όσα περισσότερα rides μπορεί, με βάση τους επιβάτες που περιμένουν στο queue του. Όταν ειδοποιήσουμε από την main ότι δεν υπάρχουν άλλοι επιβάτες, το τρενάκι ολοκληρώνει όλα τα rides που απομένουν και μετά εξέρχεται της ρουτίνας του.

Το κάθε thread-επιβάτης περιμένει να επιβιβαστεί και όταν ο τελευταίος παρατηρήσει ότι το τρένο έχει γεμίσει ειδοποιεί το τρένο να ξεκινήσει το τρέχον ride.

Λύνουμε τα race conditions με χρήση των σηματοφόρων. Μέσω του mutex προστατεύουμε τις κρίσιμες μεταβλητές όπως το queue, το capacity και το num.

Λειτουργία main

Δίνουμε στο main thread έναν αριθμό επιβατών για να γεμίσει το queue. Όταν στείλουμε EOF η main ειδοποιεί το τρένο να ολοκληρώσει όλα τα πλήρη rides που απομένουν με βάση το queue και στη συνέχεια να τερματίσει.

```
while(true)
{
    if(scanf(temp) == EOF)    break;
    if(temp < 0 || temp > cap)
    {
        printf("Error!");
        continue;
    }
    else    // create passenger
        for(i = 0; i < temp; ++i)
            pthread_create(passenger);
}
```


Λειτουργία των passenger threads

Οι επιβάτες μπαίνουν στο τρενάκι ένας ένας και ο τελευταίος (εκείνος που θα ανιχνεύσει ότι το τρενάκι έχει γεμίσει, ξυπνάει το train thread για να ξεκινήσει το τρέχον ride.

```
down(mutex);
queue++;
up(mutex);
down(pass_board); // wait for boarding
if(!on && queue < capacity - num)
{
    down(mutex);
    queue--;
    up(mutex);
    up(pass_board);
    return NULL;
}

down(mutex);
num++;
queue--;
up(mutex);

if(num == capacity)
    up(train_go); // if train full, last passenger invokes train ride
else
    up(pass_board); // else allow next passenger to board
```

Λειτουργία του train thread

Το τρενάκι περιμένει να γεμίσει και όταν η main το ειδοποιήσει να τερματίσει, λέει στους εναπομείναντες επιβάτες στο queue να τερματίσουν και αυτοί.

```
while(true)
{
    down(train_go);
    if(!on && queue < capacity) { up(pass_board); break; }

    sleep(DUR_FLIGHT);

    down(mutex);
    num = 0;
    up(mutex);
    up(pass_board);    // signal to blocked passengers that they can board
}
```